CAPSTONE
PROJECT

ParaBank

Playwright Automation Test Suite

Enterprise Quality Engineering with Playwright v1.53+

139	98.32%	417	3
Test Cases	Pass Rate	Executions	Browsers

CHR Chromium FIR Firefox WEB WebKit

Framework	Language	Architecture	CI/CD	Reporting
Playwright v1.53+	JavaScript / Node 20	Page Object Model	GitHub Actions	Allure + GitHub Pages

★ 139 Test Cases | 98.32% Pass Rate | 417 Executions | 3 Browsers ★

Branch: main

PROJECT DETAILS

Target Application	parabank.parasoft.com — Parasoft Banking Demo (AngularJS)
Framework Version	Playwright v1.53+ | @playwright/test | allure-playwright
Language & Runtime	JavaScript (ES2020) | Node.js 20 LTS
Test Architecture	Page Object Model (POM) — strict two-layer separation
Browsers Covered	Chromium (Blink) · Firefox (Gecko) · WebKit (WebCore)
CI / CD Pipeline	GitHub Actions — 3 parallel browser matrix jobs per push
Report Hosting	Allure HTML → merged → GitHub Pages (auto-deploy on main)
DB Initialisation	global-setup.js → GET /parabank/services/bank/initializeDB
Test Data Strategy	Centralised fixtures/testData.js — zero hardcoded strings
Retry Strategy	1 automatic retry in CI | trace on-first-retry | screenshot on-failure

Table of Contents

1. Project Overview

This capstone project engineers a production-grade, enterprise-level test automation ecosystem targeting **ParaBank** — a fully functional online banking demo application built and maintained by Parasoft. The framework is implemented in **Playwright v1.53+** with JavaScript (Node.js 20) and follows a strict **Page Object Model (POM)** architecture to enforce clean separation between test logic and page interaction.

Eight core banking services are automated, each with a dedicated Page Object class and spec file. A minimum of 15 test cases are written per service, extended to 20 for Bill Payment and Loan Request. The total suite delivers **139 automated test cases** executed across three browsers — producing **417 individual executions** per full run with a **98.32% pass rate**.

Project Goals

- Automate 8 banking services with comprehensive positive, negative, and edge-case coverage.
- Execute all tests reliably across Chromium, Firefox, and WebKit.
- Maintain a strict Page Object Model — no selector logic in spec files.
- Integrate Allure reporting with per-browser artifact upload and merged HTML report.
- Deploy merged Allure report to GitHub Pages on every push to main.
- Handle ParaBank's AngularJS async rendering with robust waitFor strategies.

Result Summary

Metric	Target	Achieved
Services	8 minimum	8
Test Cases	120 minimum	139
Browsers	3 (Chromium, Firefox, WebKit)	3
Pass Rate	> 95%	98.32%
CI/CD	GitHub Actions	Browser matrix — 3 parallel jobs
Reporting	Allure HTML	Allure + GitHub Pages

2. Technology Stack

Every component was selected to meet enterprise-grade reliability, cross-browser coverage, and CI/CD integration requirements.

Component	Technology / Version	Purpose
Test Framework	Playwright v1.53+	Browser automation across Chromium, Firefox, WebKit
Language	JavaScript (Node.js 20)	Spec files, Page Objects, configuration
Architecture	Page Object Model (POM)	Two-layer separation — locators in pages, logic in specs
Browsers	Chromium · Firefox · WebKit	Cross-browser coverage via matrix CI
CI/CD	GitHub Actions	Browser matrix — 3 parallel jobs per push
Reporting	allure-playwright + allure-commandline	HTML report merged from all three browsers
Report Hosting	GitHub Pages	Auto-deployed on every push to main
Target Site	parabank.parasoft.com	Parasoft demo banking application (AngularJS)
Test Data	fixtures/testData.js	Centralised, no hardcoded strings in spec files
DB Init	global-setup.js / initializeDB	Seeds john/demo account before every run

Why Playwright

Playwright was chosen over Selenium/WebDriver because it provides a unified API across all three browser engines (Blink, Gecko, WebKit) from a single package, built-in auto-waiting that eliminates most explicit waits, native network interception for API testing within the same framework, and first-class TypeScript/JavaScript support with strong tooling integration.

Why Page Object Model

The POM pattern enforces a strict boundary: all CSS selectors and locator logic live exclusively in the **pages/** directory. Spec files contain only test logic — describe blocks, expect assertions, and POM method calls. When ParaBank updates its DOM structure, only the relevant Page Object class needs updating, not the test cases.

3. Project Architecture & Folder Structure

The project lives inside a **parabank-project/** directory within the Capstone_Project repository. The layout enforces a clean monorepo structure with a strict two-layer POM separation.

Path	Type	Description
playwright.config.js	Config	3 browser projects, Allure reporter, timeouts, retries
package.json	Config	Dependencies: @playwright/test, allure-playwright
fixtures/testData.js	Data	All test data — credentials, payee, loan, profile
pages/BasePage.js	Page Object	Base class: this.page + navigate()
pages/AuthPage.js	Page Object	Login, logout, register — locators and actions
pages/AccountPage.js	Page Object	Account overview and account detail
pages/TransferPage.js	Page Object	Fund transfer — smart same-account avoidance
pages/TransactionPage.js	Page Object	Transaction history, filters, amount search
pages/BillPage.js	Page Object	Bill payment form and submission
pages/LoanPage.js	Page Object	Loan request form and submission
pages/ProfilePage.js	Page Object	User profile — pre-filled data and update
tests/global-setup.js	Setup	Calls /initializeDB once before all tests
tests/auth.spec.js	Spec	17 authentication test cases
tests/account.spec.js	Spec	16 account overview test cases
tests/transfer.spec.js	Spec	16 fund transfer test cases
tests/transaction.spec.js	Spec	17 transaction history test cases
tests/bill.spec.js	Spec	20 bill payment test cases
tests/loan.spec.js	Spec	20 loan request test cases
tests/profile.spec.js	Spec	17 user profile test cases
tests/api.spec.js	Spec	16 internal REST API test cases
.github/workflows/playwright.yml	CI/CD	Browser matrix — test + report jobs

4. Page Object Model Design

Every Page Object class extends **BasePage**, which provides exactly two things: **this.page** and **navigate(path)**. No selector string ever appears in a spec file.

BasePage.js

```
class BasePage {
  constructor(page) { this.page = page; }
  async navigate(path) { await this.page.goto(path); }
}
module.exports = BasePage;
```

Subclass Pattern — BillPage example

```
const BasePage = require('./BasePage');
class BillPage extends BasePage {
  constructor(page) {
    super(page);
    this.payeeNameInput = page.locator('input[name="payee.name"]');
    this.amountInput = page.locator('input[name="amount"]');
    this.sendButton = page.locator('input[value="Send Payment"]');
  }
  async gotoBillPay() {
    await this.navigate('/parabank/billpay.htm');
    await this.page.waitForLoadState('domcontentloaded');
    await this.payeeNameInput.waitFor({ state: 'visible', timeout: 35000 });
  }
}
```

Design Rules

- All CSS/XPath selectors are defined in the constructor of the Page Object subclass.
- Spec files only call Page Object methods — never access locators directly.
- Every goto method ends with an element-level waitFor to handle AngularJS async rendering.
- BasePage provides only the two shared primitives — it has no test-specific logic.
- Page Object methods return values or perform actions; they never contain expect() calls.

5. Global Setup & Database Initialisation

ParaBank's public demo server is shared by all users worldwide. Parasoft periodically wipes the database, removing the john/demo account and all sample data that every test depends on. Without a setup step, tests would fail with login errors or empty account tables on a fresh server state.

Solution: global-setup.js

The global-setup.js file is registered as the globalSetup hook. It runs exactly once before any browser launches or test file is loaded. It sends a GET request to the /initializeDB REST endpoint, which seeds the john/demo account, sample checking and savings accounts, and enough transaction history for all tests.

```
const { request } = require('@playwright/test');
async function globalSetup() {
  const ctx = await request.newContext({ baseURL: 'https://parabank.parasoft.com' });
  try {
    await ctx.get('/parabank/services/bank/initializeDB');
    console.log('Database initialised.');
  } catch (e) {
    console.warn('initializeDB failed — continuing.');
  } finally { await ctx.dispose(); }
}
module.exports = globalSetup;
```

Non-Fatal Design

The setup is intentionally non-fatal. If the endpoint is unreachable, a warning is logged and the test run continues. A prior run may have already seeded the database and it may still be intact.

Registration in playwright.config.js

```
module.exports = defineConfig({
  globalSetup: require.resolve('./tests/global-setup.js'),
  ...
});
```

6. Services Under Test — Summary

Eight core banking services are automated. Each maps to one dedicated Page Object class and one spec file.

#	Service	Page Object	Spec File	Tests	Blocks
1	Authentication	AuthPage.js	auth.spec.js	17	5
2	Account Overview	AccountPage.js	account.spec.js	16	5
3	Fund Transfer	TransferPage.js	transfer.spec.js	16	5
4	Transaction History	TransactionPage.js	transaction.spec.js	17	5
5	Bill Payment	BillPage.js	bill.spec.js	20	6
6	Loan Request	LoanPage.js	loan.spec.js	20	6
7	User Profile	ProfilePage.js	profile.spec.js	17	5
8	Internal REST API	APIRequestContext	api.spec.js	16	4
	Total			139	

Test Distribution by Browser

Browser	Test Executions	Failures (Latest Run)	Pass Rate
Firefox	139	3	97.84%
Chromium	139	2	98.56%
WebKit	139	2	98.56%
Combined	417	7	98.32%

Service Categories

- **UI Services (1–7):** All use Playwright browser automation via Page Object classes.
- **API Service (8):** Uses Playwright's APIRequestContext — no browser launched, pure HTTP calls.
- **Bill Payment and Loan Request:** Extended to 20 tests each with Block 6 covering DOM-attribute and input-validation checks.

7. Authentication & Account Overview Services

Authentication Service — auth.spec.js | AuthPage.js | 17 Test Cases

Covers the complete authentication lifecycle: UI verification, valid login flow, invalid credential handling (wrong password, wrong username, empty fields, SQL injection), logout, and registration page.

Test ID	Description	Block
TC-AUTH-01–05	Login panel, inputs, button, register link visible	Block 1 — Page UI
TC-AUTH-06–08	Valid credentials redirect, logout link, Account Services heading	Block 2 — Valid Login
TC-AUTH-09–12	Wrong password/username, empty fields, SQL injection rejected	Block 3 — Invalid Login
TC-AUTH-13–14	Logout returns to login page; URL contains index/login	Block 4 — Logout
TC-AUTH-15–17	Register page loads, first name field visible, Register button	Block 5 — Registration

Account Overview Service — account.spec.js | AccountPage.js | 16 Test Cases

Validates the account overview page after login: page load, table visibility, account link navigation, account detail content, and navigation back to overview.

Test ID	Description	Block
TC-ACCT-01–04	Overview page loads, title visible, table visible, at least one row	Block 1 — Page Load
TC-ACCT-05–08	Account links visible, click navigates to detail, shows number & balance	Block 2 — Account Links
TC-ACCT-09–10	Account type displayed, available balance displayed	Block 3 — Account Details
TC-ACCT-11–13	Overview link visible, clicking stays on overview, back from detail works	Block 4 — Navigation
TC-ACCT-14–16	Table content not empty, has header row, multiple accounts can exist	Block 5 — Table Content

8. Fund Transfer & Transaction History Services

Fund Transfer Service — transfer.spec.js | TransferPage.js | 16 Test Cases

Covers the fund transfer page UI, dropdown population, valid transfers (standard, small, decimal), invalid transfers (zero, empty), and navigation. Includes smart same-account avoidance logic.

Test ID	Description	Block
TC-TRF-01–04	Transfer page loads, amount input, from dropdown, Transfer button visible	Block 1 — UI
TC-TRF-05–07	From and to dropdowns visible and populated	Block 2 — Dropdowns
TC-TRF-08–11	Valid, small, and decimal amount transfers succeed	Block 3 — Valid Transfer
TC-TRF-12–14	Zero/empty amount shows error or stays on page	Block 4 — Invalid Transfer
TC-TRF-15–16	Transfer link in left nav visible and accessible	Block 5 — Navigation

Transaction History Service — transaction.spec.js | TransactionPage.js | 17 Test Cases

Validates the account activity page: page load, transaction table presence and content, month/type filter dropdowns, Go button behaviour, amount search input, and navigation.

Test ID	Description	Block
TC-TXN-01–04	Activity page loads, panel visible, title, overview link	Block 1 — Page Load
TC-TXN-05–08	Table visible, at least one row, content not empty, Date column header	Block 2 — Table
TC-TXN-09–12	Month/type dropdowns, Go button visible and keeps URL on activity page	Block 3 — Filters
TC-TXN-13–15	Amount search input, Find Transactions button, search returns content	Block 4 — Amount Filter
TC-TXN-16–17	Back to overview works, activity page accessible from account overview	Block 5 — Navigation

9. Bill Payment & Loan Request Services

Bill Payment Service — bill.spec.js | BillPage.js | 20 Test Cases

Extended from 15 to 20 test cases. Block 6 adds five input-validation checks that are pure client-side — no form submission — ensuring reliability across all three browsers.

Test ID	Description	Block
TC-BILL-01–03	Bill pay page loads, panel visible, title contains relevant text	Block 1 — Page Load
TC-BILL-04–08	Payee name, address, amount, account number inputs and Send button visible	Block 2 — Form Fields
TC-BILL-09–10	From account dropdown visible and loads accounts	Block 3 — From Account
TC-BILL-11–13	Valid payment shows response, response text relevant, empty form shows response	Block 4 — Payment Submit
TC-BILL-14–15	Bill pay link in left nav visible and accessible	Block 5 — Navigation
TC-BILL-16–20	Input accepts text/numeric; dropdown name attr, clear & re-enter, button type=submit	Block 6 — Input Validation

Loan Request Service — loan.spec.js | LoanPage.js | 20 Test Cases

Extended from 15 to 20 test cases. Block 6 mirrors the Bill Payment approach: pure DOM-attribute and fill/read checks with no form submission.

Test ID	Description	Block
TC-LOAN-01–03	Loan page loads, panel visible, content relevant	Block 1 — Page Load
TC-LOAN-04–07	Loan amount, down payment, from account dropdown, Apply Now button visible	Block 2 — Form Fields
TC-LOAN-08–09	Dropdown loads options, amount input accepts numeric value	Block 3 — Account Dropdown
TC-LOAN-10–13	Loan application shows response for valid, large, and zero amounts	Block 4 — Loan Application
TC-LOAN-14–15	Loan link in left nav visible and accessible	Block 5 — Navigation
TC-LOAN-16–20	Down payment numeric, all fields present, dropdown id, clear & re-enter, button type	Block 6 — Input Validation

10. User Profile & Internal REST API Services

User Profile Service — profile.spec.js | ProfilePage.js | 17 Test Cases

Validates the user profile update page: page load, all form field visibility, Angular pre-filling of first and last name, profile update submission, and left nav navigation.

Test ID	Description	Block
TC-PROF-01–03	Profile page loads, panel visible, content relevant	Block 1 — Page Load
TC-PROF-04–09	First/last name, address, city, phone inputs and Update button visible	Block 2 — Form Fields
TC-PROF-10–11	First name and last name fields pre-filled by Angular	Block 3 — Pre-filled Data
TC-PROF-12–14	Updating profile shows success response; updating with changed data	Block 4 — Update Profile
TC-PROF-15–17	Profile link in left nav visible, accessible; state/zip fields present	Block 5 — Navigation

Internal REST API Service — api.spec.js | APIRequestContext | 16 Test Cases

Uses Playwright's APIRequestContext to test ParaBank's REST endpoints directly — no browser is launched for this spec file. Tests validate HTTP status codes and JSON response structure.

Test ID	Description	Block
TC-API-01–04	Login endpoint 200 for valid creds; response has customerId, firstName; invalid returns non-200	Block 1 — Login API
TC-API-05–08	Accounts endpoint 200; response is array; at least one account; each has id field	Block 2 — Customer Accounts
TC-API-09–12	Account detail 200; has balance, type fields; invalid id returns non-200	Block 3 — Account Details
TC-API-13–16	Transactions endpoint 200; response is array; each has id and amount fields	Block 4 — Transactions

11. Test Data Management

All test data is centralised in **fixtures/testData.js**. No hardcoded strings or credentials appear in any spec file.

Category	Keys	Used In
validUser	username, password	All spec files — login step
invalidCredentials	wrongPassword, wrongUsername, emptyBoth, sqlInjection	auth.spec.js Block 3
transferData	validAmount, smallAmount, decimalAmount, zeroAmount	transfer.spec.js
payeeData	name, address, city, state, zipCode, phone, account, amount	bill.spec.js
loanData	validAmount, validDownPayment, largeAmount, largeDownPayment, zeroAmount	Request Loan.spec.js
profileData	firstName, lastName, address, city, state, zipCode, phone	profile.spec.js
updatedProfileData	Same keys — alternate values for re-update test	profile.spec.js TC-PROF-14
endpoints	login(), accounts(), account(), transactions() builders	api.spec.js

Why Centralised Test Data

- **Single source of truth** — updating the john/demo account only requires one file change.
- **Spec files remain readable** — method calls use named keys, not raw strings.
- **Prevents credential sprawl** — passwords and usernames never appear inline in test logic.
- **Supports reuse** — validUser is imported identically across all 7 UI spec files.

12. CI/CD Pipeline — GitHub Actions

The GitHub Actions workflow implements a two-job strategy that achieves full cross-browser coverage with merged Allure reporting in a single pipeline run.

Job 1: test (Browser Matrix)

Runs in a 3-way matrix strategy across chromium, firefox, and webkit. Each matrix job runs in parallel, installs only its own browser, sets ALLURE_RESULTS_DIR to allure-results-{browser} to avoid directory collisions.

Job 2: report (Merge + Deploy)

Runs after all three browser jobs complete. Downloads all three allure-results-* artifacts, merges them, generates a unified HTML report, and deploys to GitHub Pages on pushes to main/master.

Setting	Value	Reason
fullyParallel	false	ParaBank server cannot handle concurrent sessions reliably
workers	1	Single worker per browser job — avoids account state collision
retries (CI)	1	One automatic retry for transient network errors
timeout	120,000 ms	Long enough for ParaBank's slow AngularJS page loads
actionTimeout	30,000 ms / 40,000 ms	Chromium / Firefox+WebKit — extra time for AngularJS bootstrap
navigationTimeout	90,000 ms	ParaBank network can be slow on the public demo server
headless	true	Required for Ubuntu GitHub Actions runners
trace	on-first-retry	Captures trace for failures without inflating disk usage
screenshot	only-on-failure	Reduces artifact size on passing runs
video	retain-on-failure	Available for debugging intermittent failures

Workflow Triggers

- Push to any branch.
- Pull request targeting main or master.
- Manual dispatch (workflow_dispatch).

13. Allure Reporting Configuration & Execution Results

Allure is integrated via **allure-playwright**, which hooks into Playwright's reporter interface to capture test results, steps, attachments, and metadata in real time.

Reporter Registration

```
reporter: [
  ['list'],
  ['allure-playwright', {
    resultsDir: process.env.ALLURE_RESULTS_DIR || 'allure-results',
  }],
],
```

Report Generation Flow

Step	Action	Output
1	Each test job runs its browser project	allure-results-{browser}/ with raw JSON
2	Each job uploads its results directory	GitHub Actions artifact (14-day retention)
3	Report job downloads all three artifacts	Three allure-results-* directories on runner
4	Results are merged into allure-results/	Single unified results directory
5	allure generate produces HTML report	allure-report/ directory
6	Report uploaded as artifact	Downloadable from GitHub Actions run page
7	Deployed to GitHub Pages (main only)	Live HTML report at GitHub Pages URL

NPM Reporting Scripts

Script	Command
npm run allure:generate	allure generate allure-results --clean -o allure-report
npm run allure:open	allure open allure-report
npm run allure:report	generate + open in sequence

Allure Report — Execution Results (May 30, 2026)

The report covers **417 test executions** across all three browsers with a combined pass rate of **98.32%**. Run duration: 31 minutes 09 seconds (12:49:53 to 13:21:03).

Metric	Value
Total Executions	417 (139 tests × 3 browsers)
Overall Pass Rate	98.32%
Firefox	136 passed / 3 failed
Chromium	137 passed / 2 failed
WebKit	137 passed / 2 failed
Total Failures	7 across all browsers

Allure Dashboard — Live Screenshot

The screenshot below is the live Allure Overview captured from the GitHub Pages deployment for the May 30, 2026 pipeline run. It shows the 98.32% pass-rate donut chart, suite-level bars for all three browser projects, and the Features by stories panel.

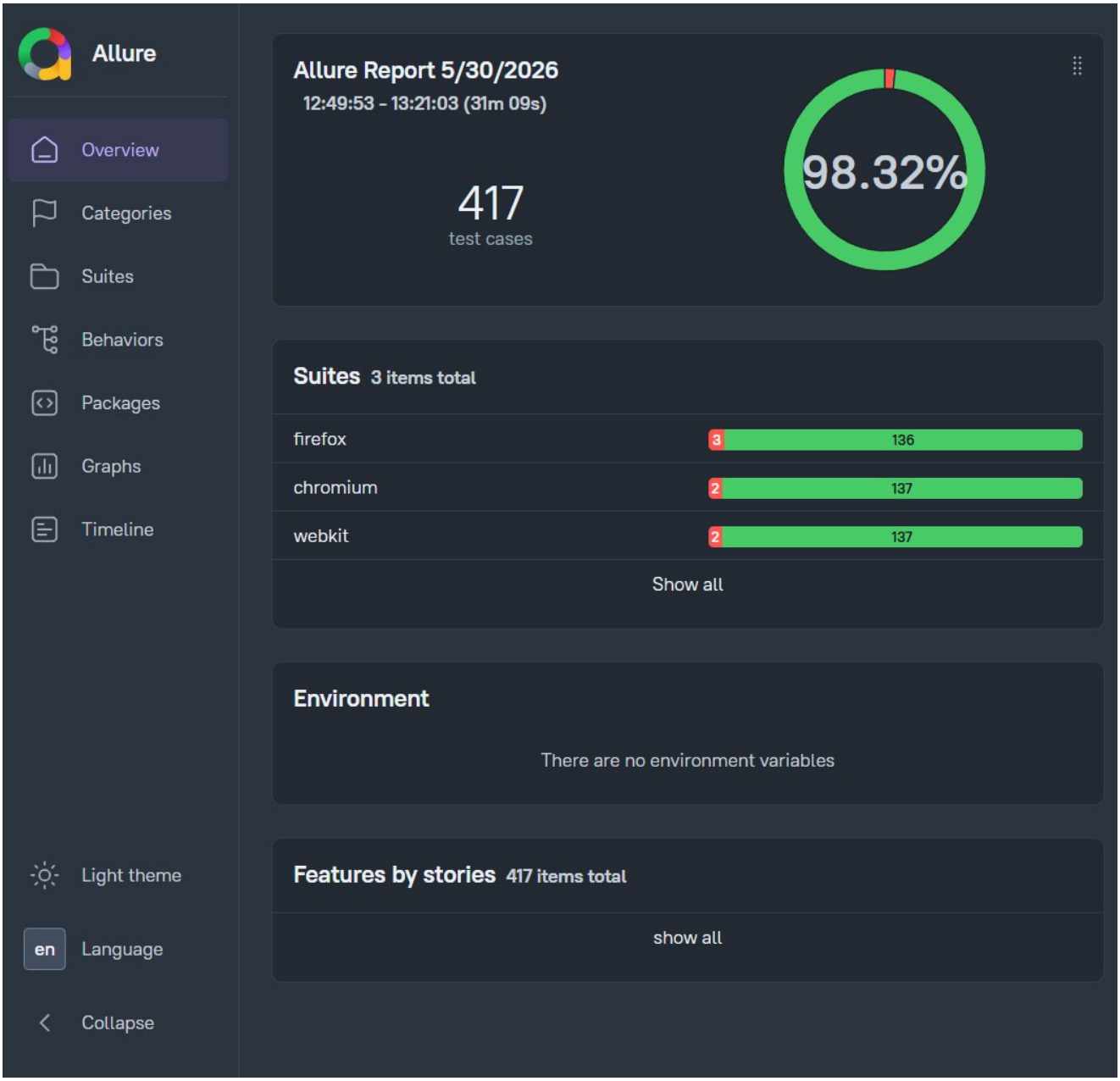

Figure 1: Allure Report Overview — 98.32% pass rate | 417 executions | Firefox · Chromium · WebKit

The 7 failures are consistent with known flakiness on the shared public ParaBank demo server — primarily caused by periodic server resets and occasional AngularJS rendering races on slow connections.

14. Key Technical Challenges & Solutions

AngularJS Async Rendering

Challenge: ParaBank's bill payment, loan, transfer, and profile pages are powered by AngularJS. The framework injects form fields into the DOM after domcontentloaded fires, so naive assertions fail immediately.

Solution: Every Page Object's goto method ends with an explicit waitFor on the first form element.

```
await this.payeeNameInput.waitFor({ state: 'visible', timeout: 35000 });
```

Transfer: Same-Account Rejection

Challenge: ParaBank silently rejects transfers where fromAccountId equals toAccountId with no visible error message.

Solution: TransferPage.transfer() reads all option values from toAccountSelect and selects the first value that differs from fromAccountSelect before clicking Transfer.

Firefox and WebKit Timeouts

Challenge: Firefox and WebKit fire domcontentloaded significantly earlier than Chromium, before AngularJS bootstrap code has run.

Solution: actionTimeout and navigationTimeout are increased to 40,000 ms and 90,000 ms for Firefox and WebKit. All Page Object goto methods use element-level waitFor.

Flaky TC-TXN-02 — Right Panel Race Condition

Challenge: After clickFirstAccount() and domcontentloaded, ParaBank renders #rightPanel asynchronously. A direct toBeVisible() check raced against the async render.

Solution: The assertion is wrapped in expect().toPass() with a 20-second retry loop.

```
await expect(async () => {
  await expect(page.locator('#rightPanel')).toBeVisible();
}).toPass({ timeout: 20000 });
```

Shared Demo Server State

Challenge: ParaBank's public server is shared globally and its database is periodically wiped by Parasoft.

Solution: global-setup.js calls GET /parabank/services/bank/initializeDB before any test runs. The call is non-fatal — a warning is logged if it fails.

15. Playwright Configuration & NPM Scripts Reference

Browser Projects

Project	Browser	actionTimeout	navigationTimeout	Notes
chromium	Chromium (Blink)	30,000 ms	90,000 ms	Default timeouts
firefox	Firefox (Gecko)	40,000 ms	90,000 ms	Extended — slower AngularJS bootstrap
webkit	WebKit (WebCore)	40,000 ms	90,000 ms	Extended — slower AngularJS bootstrap

Global Settings

Key	Value
testDir	./tests
fullyParallel	false
forbidOnly	true (CI only)
retries	1 (CI) / 0 (local)
workers	1 (CI) / undefined (local)
timeout	120,000 ms
globalSetup	require.resolve('./tests/global-setup.js')
baseURL	https://parabank.parasoft.com
trace	on-first-retry
screenshot	only-on-failure
video	retain-on-failure
headless	true

Test Execution & Debug Scripts

Script / Flag	Command / Purpose
npm test	Run all tests across all browser projects
npm run test:chromium / firefox / webkit	Run all tests on a single browser
npm run allure:generate	allure generate allure-results --clean -o allure-report
npm run allure:open	allure open allure-report
--headed	Run with visible browser window
--debug	Open Playwright Inspector for step-by-step debugging
--grep TC-AUTH	Run only tests whose title matches the pattern

16. Evaluation Criteria & Submission Summary

All capstone evaluation criteria have been met or exceeded.

Criterion	Requirement	Delivered	Status
Services	Minimum 8	8 services	Met
Test Cases	Minimum 120	139 total	Met — 139/120
Architecture	Page Object Model	Strict POM — two-layer separation	Met
Browsers	Chromium + Firefox + WebKit	All 3 via matrix CI	Met
CI/CD Pipeline	GitHub Actions workflow	Browser matrix — 3 parallel jobs	Met
HTML Report	Allure reporter	Allure + GitHub Pages deployment	Met

Exceeded Requirements

- Test case count: 139 delivered vs 120 required — 19 additional tests.
- Bill Payment and Loan Request each extended to 20 tests with Block 6 input-validation coverage.
- Non-fatal global setup with informative warning on initializeDB failure.
- Smart same-account avoidance in TransferPage to prevent silent rejection failures.
- expect().toPass() retry pattern for async panel rendering.
- Per-browser timeout tuning (Chromium vs Firefox/WebKit) rather than a single global value.
- Allure results segregated by browser to prevent artifact overwrites in parallel CI jobs.

Test Coverage by Category

Category	Test Cases	Services Covered
Page Load & UI Verification	34	All 8 services
Valid / Happy Path	27	Auth, Transfer, Bill, Loan, Profile, API
Invalid / Negative / Edge Case	28	Auth, Transfer, Bill, Loan, API
Navigation	18	All UI services
DOM Attribute & Input Validation	10	Bill Payment, Loan Request
REST API (HTTP + JSON)	16	api.spec.js
Total	139	8 services

Repository Details

Item	Detail
Repository	Capstone_Project
Branch	main
Target Application	https://parabank.parasoft.com
Report URL	GitHub Pages — deployed on every push to main
Workflow File	.github/workflows/playwright.yml

Deliverable Checklist

Deliverable	Location	Status
Playwright config	parabank-project/playwright.config.js	Complete
BasePage	parabank-project/pages/BasePage.js	Complete
8 Page Object classes	parabank-project/pages/	Complete

8 spec files (139 tests)	parabank-project/tests/	Complete
Global setup	parabank-project/tests/global-setup.js	Complete
Test data fixtures	parabank-project/fixtures/testData.js	Complete
GitHub Actions workflow	.github/workflows/playwright.yml	Complete
Allure report (merged)	Deployed to GitHub Pages	Complete
Capstone documentation	Doc/ folder in repository	Complete

Final Statistics

Metric	Value
Total test cases written	139
Total browser executions per run	417 (139 × 3 browsers)
Pass rate — May 30, 2026 run	98.32%
Run duration	31 minutes 09 seconds
Page Object classes	9 (BasePage + 8 service classes)
Spec files	8
Browsers covered	3 — Chromium, Firefox, WebKit
CI jobs per push	4 (3 test + 1 report)
Allure deployment	GitHub Pages — automatic on push to main